Running the Collision Logic on the Heltec Wireless Tracker

PS2 Automated TCAS for General Aviation | Devils Invent 2026

Yes. The "brain" on the Heltec Wireless Tracker is an **ESP32-S3**: a dual-core processor running up to 240 MHz with hardware floating-point support and about 512 KB of working memory (RAM). That's enough to run these calculations, but **in the demo, the board is not the one doing them.**

What's doing the calculations right now

In the recordings, everything runs **in Python on the laptop**:

Part of the demo	Technology
Flight physics of each Skyhawk	JSBSim (C++ flight model, called from Python)
Collision logic: τ, closest approach, uncertainty, maneuver search	Python with NumPy, which does the 40-sample simulations as fast array math
Packets and negotiation	Python, producing the real 33-byte format with CRC checks
Dashboard, audio, explosion	Pygame
3D view	FlightGear

The laptop is **simulating what each Heltec board would compute**. When presenting, say: "This is the onboard software running in simulation; the same logic is sized to fit on the ESP32-S3 on each board." Don't say the boards are running it unless that has actually been done.

Could the ESP32-S3 run it?

Yes, as long as the code is written for a small chip rather than copied as-is. These are rough estimates.

The easy parts (well under 1% of the chip's time):

- Encoding and decoding packets, plus the checksum
- Tracking each plane and predicting where it's going
- τ, closest approach, the alert tests

The demanding part, the maneuver search:

Case	Work per decision	Estimated time on ESP32-S3
2 aircraft, written as-is	25 plans × 40 samples × ~180 time steps	~0.1–0.2 s
5 aircraft, written as-is	~1,000 plan combinations	~0.4 s
Optimized (fewer samples, no repeated trig)	same results	well under 0.1 s

The decision cycle runs once per second, and the plan is agreed during the traffic advisory, about 20 seconds before it's needed, so there's plenty of slack. The second processor core can run the search while the first handles the radio and GPS timing.

Two real constraints to plan for:

1. **Memory.** The Python version stores every simulated path at once, which would take several megabytes. The ESP32-S3 has about 512 KB of working memory. The fix is to compute each plan's closest approach while simulating it, instead of storing whole paths. That's a normal rewrite for embedded code, not a redesign.
2. **Language.** NumPy and this Python code won't run on the board. The real version would be **C++** (Arduino or Espressif's ESP-IDF toolkit), using existing libraries:
   - RadioLib for the SX1262 LoRa radio
   - an NMEA parser for the UC6580 GPS
   - TFT_eSPI or LovyanGFX for the small display
   - FreeRTOS (built into the ESP32) to split work between the two cores

What the board can't do

The board's display is only 0.96 inches (160×80 pixels), so it shows the advisory ("TURN RIGHT"), a simple traffic arrow, and status, not the full dashboard. The dashboard in the demo is an engineering view of what the board is thinking. Audio needs an external buzzer or a connection to the aircraft's audio panel.

How to answer if a judge asks

> *"In the demo the onboard logic runs in Python so we can visualize every step. The real target is the ESP32-S3 on the Heltec board. The heavy part is the maneuver search, which we estimate at about 0.1–0.4 seconds per decision on that chip as written, and well under 0.1 seconds after optimization, against a 1-second decision cycle with the plan agreed about 20 seconds in advance. The next step is porting it to C++ and running it on two real boards with the simulator feeding them GPS data."*

That last sentence describes the strongest next milestone: **hardware-in-the-loop**. The laptop runs the flight simulation and sends each board fake GPS data over USB. The boards exchange real LoRa packets and run the real collision logic, then send their maneuver commands back to the simulator. That would prove the boards, not the laptop, are doing the work.